UCG
Univerzitet Crne Gore

OPERATIVNI SISTEMI

Procesi, signali i niti u C-u

fork · exec · wait · signali · POSIX niti

Velibor Došljak
10. novembar 2026.

Pregled

1. Procesi
2. Pokretanje programa: exec
3. Signali
4. Niti
5. Pregled

Uz prezentaciju

Skripta `procesi_niti.pdf` i primjeri `01_fork.c` ... `16_trka_niti.c`. Na slajdovima su samo najvažniji dijelovi; detalji su u skripti.

```
$ make                  # svi primjeri
$ gcc -pthread 01_fork.c -o 01_fork
$ man 2 fork            # priručnik
```

01

Procesi

Program i proces

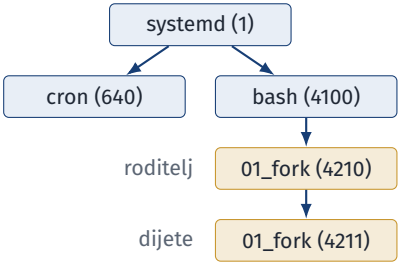
- **Program**: pasivan fajl na disku (izvršni kod)
- **Proces**: program u izvršavanju, sa svojim podacima, registrima, otvorenim fajlovima…
- Isti program može istovremeno da radi kao više procesa, npr. više terminala sa `bash`-om
- Za svaki proces jezgro vodi **kontrolni blok procesa** (PCB)

PCB: kontrolni blok procesa

PID i PPID (PID roditelja)
stanje: R, S, D, T, Z
registri i programski brojač
tabela otvorenih fajlova
korisnik (UID), prioritet
obrada signala

Stablo procesa

- Svaki proces ima **roditelja** koji ga je napravio
- Izuzetak je prvi proces, `systemd` (PID 1)
- `getpid()`: moj PID
  `getppid()`: PID mog roditelja
- Stablo u terminalu: `pstree -p`



Adresni prostor procesa

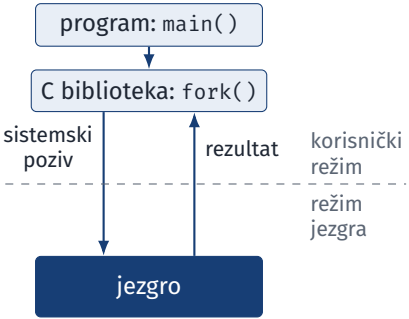


- Svaki proces ima svoj **virtuelni** adresni prostor
- Adrese koje program vidi (`printf("%p")`) su virtuelne; jezgro i MMU ih preslikavaju u fizičke
- Ista virtuelna adresa u dva procesa pokazuje na različitu fizičku memoriju
- Zato proces ne može da pokvari memoriju drugog procesa

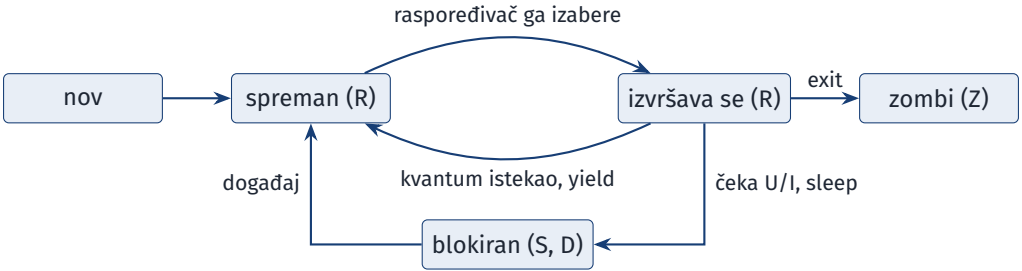
Sistemski pozivi

- Proces sam ne može da pravi procese, šalje signale ili čita disk: to radi **jezgro**
- Program to traži **sistemskim pozivom**: `fork`, `execve`, `kill`, `waitpid`...
- Greška: poziv vrati `-1` i upiše kod greške u `errno`

```
if (fork() == -1) {
    perror("fork");  // "fork: " + opis
    exit(1);
}
```



Stanja procesa



Oznake u koloni `STAT` komande `ps -o pid,stat,cmd`:

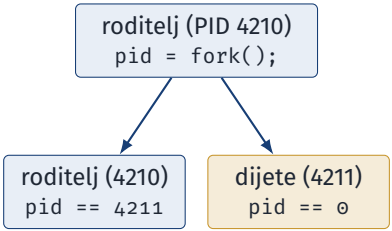
- **R** izvršava se ili je spreman
- **S** spava, čeka događaj
- **D** čeka uređaj, ne prekida se
- **T** zaustavljen (Ctrl+Z, `SIGSTOP`)
- **Z** zombi: završio, roditelj nije pozvao `wait`

fork: pravljenje novog procesa

`pid_t fork(void);` `#include <unistd.h>`

- Pravi **novi proces (dijete)**: kopiju procesa koji ga je pozvao
- Dijete nastavlja odmah **poslije** `fork()`
- Poziva se jednom, a vraća dva puta:
  - roditelju: PID djeteta (> 0)
  - djetetu: `0`
  - greška: `-1`, dijete nije napravljeno

```
pid_t pid = fork();
if (pid < 0)  { perror("fork"); exit(1); }
if (pid == 0) { /* dijete */ }
else          { /* roditelj */ }
```



isti kod, kopija podataka, nastavljaju od iste tačke

Prvi primjer

```
$ ./01_fork
Prije fork-a: PID=4210
[RODITELJ] PID=4210, PPID=4100, fork je vratio 4211
[DIJETE]   PID=4211, PPID=4210, fork je vratio 0
Kraj programa (PID=4211)
Kraj programa (PID=4210)
```

- Redoslijed roditelja i djeteta **nije određen**: odlučuje raspoređivač
- Posljednji `printf` izvršavaju **oba** procesa: poslije `if`-a njihovi putevi se spajaju
- PPID roditelja je PID shell-a iz kog je pokrenut
- Roditelj poziva `wait(NULL)`: bez toga bi dijete moglo da piše poslije prompta

Primjer: `01_fork.c`

Poslije fork-a memorija nije zajednička

```
$ ./02_adresni_prostor
[DIJETE]   globalna=110 (0x5f5e30cd8010)
[DIJETE]   lokalna=120  (0x7ffc2b22e628)
[DIJETE]   dinamicka=130 (0x5f5e5a88f010)
[RODITELJ] globalna=10 (0x5f5e30cd8010)
[RODITELJ] lokalna=20  (0x7ffc2b22e628)
[RODITELJ] dinamicka=30 (0x5f5e5a88f010)
```

- Dijete je promijenilo sve tri promjenljive, a roditelj vidi **stare** vrijednosti
- Adrese su iste, jer su virtuelne: u svakom procesu pokazuju na drugu fizičku memoriju
- Za razmjenu podataka između procesa treba IPC: pipe, signali, dijeljena memorija

Copy-on-write: kopija tek kad zatreba



- `fork` ne kopira memoriju odmah: stranice su zajedničke i označene samo za čitanje
- Pri prvom upisu jezgro napravi kopiju **samo te stranice**
- Zato je `fork` brz i za veliki proces, a `fork` + `exec` ne kopira skoro ništa

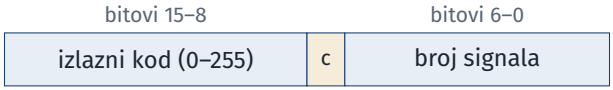
Završetak i čekanje: exit, wait, waitpid

- `exit(kod)`: kraj procesa, prazni `stdio` bafere; isto je `return kod;` iz `main`-a
- `_exit(kod)`: odmah, bez pražnjenja bafera
- `wait(&status)`: čeka **bilo koje** dijete
- `waitpid(pid, &status, opcije)`: čeka zadato dijete; `WNOHANG`: ne blokira, vrati `0` ako dijete još radi
- Vraćaju PID završenog djeteta, ili `-1` kad djece nema

```c
int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status))
    printf("kod %d\n",
           WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("signal %d\n",
           WTERMSIG(status));
```

Šta je zapisano u status



dijete je	`status`	`WIFEXITED` / `WEXITSTATUS`	`WIFSIGNALED` / `WTERMSIG`
pozvalo `exit(42)`	`0x2A00`	da / 42	ne / 0
pozvalo `exit(0)`	`0x0000`	da / 0	ne / 0
ubijeno sa `SIGKILL`	`0x0009`	ne / **0**	da / 9

Česta greška

`WEXITSTATUS` za proces ubijen signalom daje `0`, kao da je uspješno završio. Uvijek prvo `WIFEXITED` ili `WIFSIGNALED`, pa tek onda odgovarajući makro.

Zombi i siroče

Zombi

Dijete je završilo, a roditelj nije pozvao `wait`: ostaje samo zapis u tabeli procesa. Uklanja ga tek `wait` roditelja.

Siroče

Roditelj je završio prije djeteta. Dijete usvaja PID 1, koji umjesto roditelja poziva `wait`.

```
$ ./03_wait_zombi
[DIJETE]   PID=5121, zavrsavam sa exit(42)
[RODITELJ] PID=5120, spavam 15s - dijete je sada zombi
# u drugom terminalu, dok roditelj spava:
$ ps -o pid,ppid,stat,cmd --ppid 5120
    PID    PPID STAT CMD
   5121    5120 Z+   [03_wait_zombi] <defunct>
```

Više djece: fork u petlji

```c
for (int i = 0; i < N; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Dijete %d\n", i);
        sleep(i + 1);
        exit(i);   // NE nastavlja petlju
    }
}
int status;
while (wait(&status) > 0)  // -1: kraj
    printf("kod %d\n", WEXITSTATUS(status));
```

Bez exit(i)

Dijete nastavlja petlju i pravi svoju djecu: od N krugova nastaje do 2^N procesa.

- Dijete ima svoju kopiju `i`, pa zna koje je po redu
- `wait` vraća djecu redom kojim **završavaju**

Primjer: `04_vise_procesa.c`

Pitalica: koliko puta se ispiše „Zdravo“?

```c
int main() {
    fork();
    fork();
    fork();
    printf("Zdravo\n");
    return 0;
}
```

Primjer: `05_fork_pitalica.c`

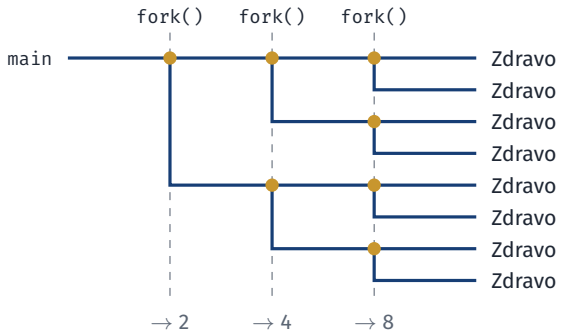
Pitalica: koliko puta se ispiše „Zdravo“?

```c
int main() {
    fork();
    fork();
    fork();
    printf("Zdravo\n");
    return 0;
}
```

Odgovor: $2^3 = 8$

Svaki proces izvršava sve `fork`-ove koji su ostali, pa se broj procesa na svakom udvostruči.

Primjer: `05_fork_pitalica.c`

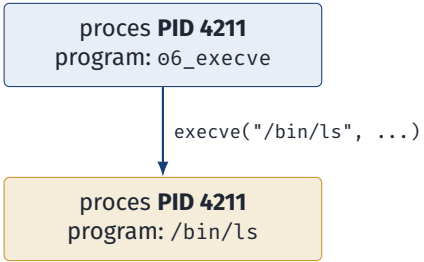


02

Pokretanje programa: exec

execve: proces postaje drugi program

- **Zamjenjuje program** koji proces izvršava: stari kod, podaci, hip i stek nestaju
- Novi program kreće od svog `main`
- PID ostaje isti, kao i otvoreni fajlovi
- Ako uspije, **nikad se ne vraća**
- Ako se vrati, desila se greška: iza `exec`-a uvijek idu `perror` i `exit`



execve u kodu

```
int execve(const char *putanja, char *const argv[], char *const envp[]);
```

```
char *argv[] = {"ls", "-l", "/tmp", NULL};
char *envp[] = {"LANG=C", NULL};

execve("/bin/ls", argv, envp);
perror("execve");     // stize se ovdje samo ako exec nije uspio
exit(1);
```

- `argv`: argumenti koje novi program dobija u svom `main`-u; `argv[0]` je po konvenciji ime programa
- `envp`: promjenljive okruženja oblika `IME=vrijednost`
- Oba niza se završavaju sa NULL: tako novi program zna koliko ih ima
- `execve` traži punu putanju i ne pretražuje `PATH`

Familija exec

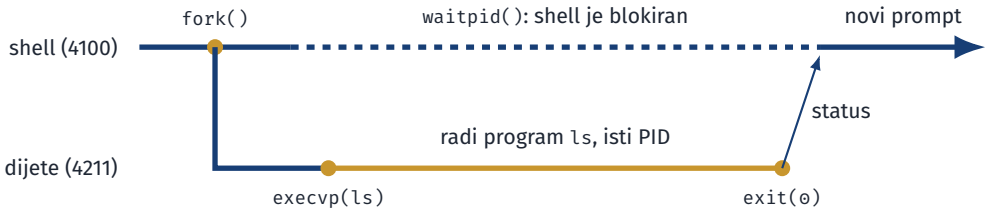
Sve su omotači oko `execve`; slova u imenu govore kako se zadaju argumenti.

slovo	značenje	primjer
l (*list*)	argumenti jedan po jedan	`execl("/bin/ls", "ls", "-l", NULL)`
v (*vector*)	argumenti kao niz	`execv("/bin/ls", argv)`
p (*path*)	traži program u `PATH`	`execlp("ls", "ls", "-l", NULL)`
e (*environment*)	zadaje se okruženje	`execve("/bin/ls", argv, envp)`

PID se ne mijenja: program postaje `ps` i u spisku vidi isti PID.

```
$ ./07_exec_isti_pid
Moj PID je 4300, sad postajem 'ps'...
    PID    PPID CMD
   4100    4090 bash
   4300    4100 ps -o pid,ppid,cmd
```

fork + exec + wait: kako shell pokreće komandu



1. Shell napravi dijete: `fork`
2. Dijete se pretvori u traženi program: `exec`
3. Shell čeka da komanda završi (`wait`), pa ispiše novi prompt

Zašto ne samo `exec`? Shell bi nestao, jer bi ga zamijenio `ls`.

Mini shell

```c
while (1) {
    printf("mini-shell> ");
    fflush(stdout);
    if (!fgets(linija, 256, stdin)) break;  // Ctrl+D
    /* rastavi liniju: argv = {"ls", "-l", NULL} */
    if (strcmp(argv[0], "exit") == 0) break;

    pid_t pid = fork();
    if (pid == 0) {
        execvp(argv[0], argv);   // p: trazi u PATH
        perror(argv[0]);
        exit(127);               // nije pronadjena
    }
    waitpid(pid, &status, 0);
}
```

```
$ ./08_mini_shell
mini-shell> echo zdravo
zdravo
[PID=5301, kod=0]
mini-shell> false
[PID=5302, kod=1]
mini-shell> abc
abc: No such file or directory
[PID=5303, kod=127]
mini-shell> exit
```

Za razmišljanje: zašto `exit` mora da izvrši sam shell? Zašto `cd` ne bi radio?

Baferovani izlaz

```
$ ./01_fork | cat
Prije fork-a: PID=4210
[DIJETE]   PID=4211, PPID=4210, fork je vratio 0
Kraj programa (PID=4211)
Prije fork-a: PID=4210
[RODITELJ] PID=4210, PPID=4100, fork je vratio 4211
Kraj programa (PID=4210)
```

- `printf` ne piše odmah, nego u **bafer** u memoriji procesa. Na terminalu se bafer prazni na kraju reda, a u pipe ili fajl tek kad se napuni ili na `exit`
- `fork` kopira memoriju, pa i bafer: „Prije fork-a“ se ispiše dva puta
- `exec` briše memoriju: neispisan tekst nestaje
- Rješenje: `fflush(stdout)` prije `fork`-a i `exec`-a (vidi `07_exec_isti_pid.c`)

03

Signali



Operativni sistemi

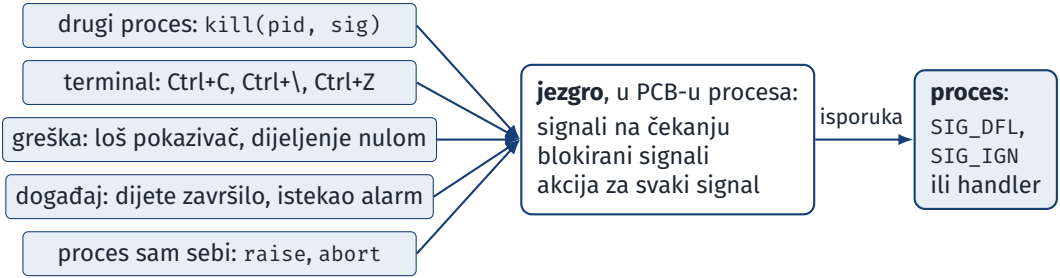
Šta je signal?

- Kratko obavještenje koje jezgro isporučuje procesu kad se desi neki događaj
- **Softverski prekid**: zaustavi proces gdje se zatekao; proces reaguje, pa nastavlja od istog mjesta
- **Asinhron**: može da stigne između bilo koje dvije instrukcije
- **Ne nosi podatke**, samo svoj broj, tj. *koji* se događaj desio
- Svaki signal ima ime i broj; u kodu se uvijek piše ime (brojevi zavise od sistema)

```
$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
```

Opis svih signala: `man 7 signal`

Ko šalje signale



Signal uvijek isporučuje **jezgro**: za proces je svejedno ko ga je poslao.

Od slanja do isporuke

Za svaki proces jezgro u PCB-u čuva:

- **signale na čekanju**: po jedan bit za svaki signal
- **masku blokiranih**: signale koje proces trenutno ne želi
- **akciju** za svaki signal

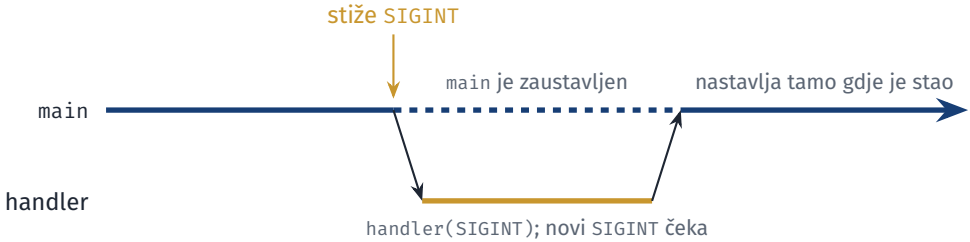
Slanje samo postavi bit. **Isporuka** se desi kad se proces vraća iz jezgra u korisnički režim. Ako proces spava (`sleep`, `read`, `wait`), jezgro ga probudi.

	SIGINT	SIGTERM	SIGUSR1
na čekanju	1	0	1
blokiran	0	0	1
ishod	isporuka	–	čeka

Signali se ne broje

Isti signal poslat tri puta prije isporuke biće isporučen **jednom**. Signal znači „desilo se bar jednom“.

Handler prekida main



- Handler se poziva **usred** `main`-a, na mjestu koje se ne može predvidjeti
- Dok handler radi, signal koji se obrađuje je automatski blokiran: novi `SIGINT` čeka
- Kad se handler vrati, `main` nastavlja kao da se ništa nije desilo

Kako proces reaguje na signal

1. **Podrazumijevana akcija** (`SIG_DFL`), zavisi od signala:
   - završetak
   - završetak + *core dump*
   - ignorisanje
   - pauza (stanje `T`) ili nastavak
2. **Ignorisanje** (`SIG_IGN`)
3. **Handler**: funkcija `void f(int sig)`

```
signal(SIGINT, SIG_IGN);  // ignorisi
signal(SIGINT, handler);  // handler
signal(SIGINT, SIG_DFL);  // zavrsetak
```

Core dump: slika memorije procesa upisana u fajl, za kasniju analizu debagerom.

Najvažniji signali

signal	br.	kada nastaje	akcija
`SIGINT`	2	Ctrl+C	završetak
`SIGQUIT`	3	Ctrl+\	završetak + core
`SIGTSTP`	20	Ctrl+Z	pauza
`SIGHUP`	1	zatvoren terminal	završetak
`SIGSEGV`	11	loš pristup memoriji	završetak + core
`SIGFPE`	8	cjelobrojno dijeljenje nulom	završetak + core
`SIGABRT`	6	`abort()`, `assert`	završetak + core
`SIGPIPE`	13	pipe koji niko ne čita	završetak

signal	br.	kada nastaje	akcija
`SIGTERM`	15	`kill PID`	završetak
`SIGKILL`	9	`kill -9 PID`	završetak*
`SIGSTOP`	19	`kill -STOP PID`	pauza*
`SIGCONT`	18	`fg`, `bg`	nastavak
`SIGUSR1`	10	slobodan	završetak
`SIGUSR2`	12	slobodan	završetak
`SIGALRM`	14	istekao `alarm()`	završetak
`SIGCHLD`	17	dijete završilo ili stalo	ignorisanje

Akcija je podrazumijevana (`SIG_DFL`). * `SIGKILL` i `SIGSTOP` se ne mogu uhvatiti, ignorisati ni blokirati. Brojevi važe za Linux na x86 i ARM procesorima.

SIGUSR1 i SIGUSR2: signali za program

- Značenje **bira program**; jezgro ih samo nikad ne šalje
- Stižu samo preko `kill()`; nov signal se ne može napraviti, ovakva su samo dva
- Podrazumijevana akcija je završetak: bez handlera `kill -USR1` ubija proces
- Primjeri: `dd` ispiše dokle je stigao, `nginx` ponovo otvori log fajlove, `10_signali_terminal.c` ispiše status

```
$ dd if=/dev/zero of=/dev/null &
[1] 6012
$ kill -USR1 6012
1039904+0 records in
1039904+0 records out
532430848 bytes (532 MB, 508 MiB) copied, 0.993603 s, 536 MB/s
```

Izlazni kod procesa ubijenog signalom

```
$ sleep 100
^C
$ echo $?
130
$ sleep 100     # pa kill -9 PID
Killed
$ echo $?
137
```

	signal	$?
Ctrl+C	2	130
Ctrl+\	3	131
`kill PID`	15	143
`kill -9 PID`	9	137

- Shell u `$?` upiše **128 + broj signala** i obično ispiše ime signala (`Killed`, `Quit`)
- To je konvencija shell-a. Roditelj u C-u isto saznaje preko `WIFSIGNALED` i `WTERMSIG`

SIGTERM ili SIGKILL

	`SIGTERM` (15)	`SIGKILL` (9)
iz terminala	`kill PID`	`kill -9 PID`
značenje	**molba** procesu: „završi se“	**naredba** jezgru: „ukloni proces“
uhvatiti, ignorisati, blokirati	može	ne može
pospremanje	proces zatvori fajlove, obriše privremene, sačuva stanje	nema ga; bafer `printf`-a se gubi
kada se koristi	uvijek prvi	tek ako proces ne reaguje na `SIGTERM`
`$?` u shell-u	143 (ako signal nije uhvaćen)	137

SIGKILL: posljednja opcija

- `SIGKILL` i `SIGSTOP` su jedini signali na koje proces ne može da utiče
- Tako je namjerno: korisnik i administrator **uvijek** moraju moći da zaustave proces, i kad je zaglavljen ili zlonamjeran
- `signal(SIGKILL, ...)` vraća `SIG_ERR`, a `errno` je `EINVAL`
- Ispravan redoslijed: `SIGTERM`, sačekati, pa tek onda `SIGKILL`. Tako rade `systemctl stop`, `docker stop` i gašenje računara

Šta kill -9 ne može

Zombi se ne može ubiti, jer je već završio: ukloni ga tek `wait` roditelja. Proces u stanju `D` (čeka disk ili drugi uređaj) primi `SIGKILL` tek kad se U/I operacija završi.

Slanje signala: kill

```
int kill(pid_t pid, int sig);
```

```
#include <signal.h>
```

`pid > 0`	proces sa tim PID-om
`pid == 0`	svi procesi iz moje grupe
`pid < -1`	svi procesi iz grupe `-pid`
`pid == -1`	svi kojima smijem da šaljem (oprezno!)
`sig == 0`	ništa se ne šalje, samo provjera da li proces postoji

Vraća `0`, ili `-1` sa `ESRCH` (nema procesa) ili `EPERM` (nema dozvole).
Proces sam sebi: `raise(sig)`, `abort()`, `alarm(n)`.

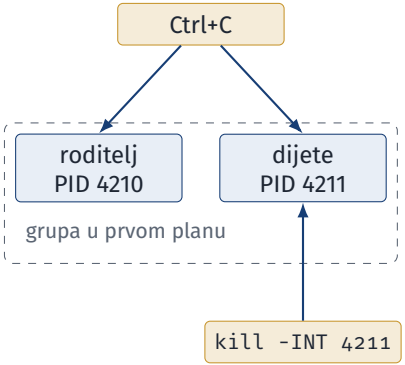
```
kill(pid, SIGTERM);   // molba
kill(pid, SIGKILL);   // naredba

if (kill(pid, 0) == 0)
    printf("proces postoji\n");
```

```
$ kill 6100          # SIGTERM
$ kill -9 6100       # SIGKILL
$ kill -USR1 6100
$ pkill 10_signali   # po imenu
```

Signali iz terminala i grupe procesa

- Za proces je **svejedno** ko je poslao signal: isti handler, isto podešavanje
- Razlika je kod slanja:
  - Ctrl+C, Ctrl+\, Ctrl+Z: cijeloj grupi procesa u prvom planu
  - `kill PID`: samo jednom procesu
  - `kill -INT -4210`: cijeloj grupi 4210
- `kill()` provjerava dozvole (isti korisnik ili `root`); signal sa terminala šalje jezgro
- Proces u pozadini (`./prog &`) ne dobija Ctrl+C



Primjer: `09_signali_kill.c`

Upravljanje poslovima (job control)

```
$ sleep 100
^Z
[1]+  Stopped                 sleep 100
$ jobs
[1]+  Stopped                 sleep 100
$ bg
[1]+ sleep 100 &
$ fg
sleep 100
^C
```

- Ctrl+Z: `SIGTSTP`, proces je pauziran (stanje `T`)
- `jobs`: pauzirani i pozadinski poslovi
- `fg`, `bg`: nastavak u prvom planu, odnosno u pozadini; oba šalju `SIGCONT`
- `./program &`: odmah u pozadini; Ctrl+C na njega ne djeluje
- Zatvoren terminal šalje `SIGHUP`; `nohup ./program &` ga ignoriše

Postavljanje obrade: signal

```
sighandler_t signal(int sig, sighandler_t akcija);
typedef void (*sighandler_t)(int);
```

- `akcija`: `SIG_DFL`, `SIG_IGN` ili funkcija `void f(int sig)`
- Vraća prethodnu akciju, ili `SIG_ERR` u slučaju greške (npr. za `SIGKILL`)
- Samo **registruje** reakciju: ne šalje signal i ne poziva handler odmah
- Ime funkcije **bez zagrada** je pokazivač na funkciju: „pozovi je kasnije“

Problem

`signal` se istorijski ponašao različito na raznim sistemima: na nekima se handler poslije prvog signala vraća na `SIG_DFL`, pa drugi Ctrl+C ubije program. Zato se koristi `sigaction`.

Postavljanje obrade: sigaction

```
int sigaction(int sig, const struct sigaction *nova, struct sigaction *stara);
```

```
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

sigaction(SIGINT,  &sa, NULL);
sigaction(SIGUSR1, &sa, NULL);
sigaction(SIGTERM, &sa, NULL);
```

Primjer: `10_signali_terminal.c`

- `sa_handler`: `SIG_DFL`, `SIG_IGN` ili funkcija
- `sa_mask`: signali **dodatno** blokirani dok handler radi
- `sa_flags`: npr. `SA_RESTART`, `SA_SIGINFO`
- `stara`: ako nije `NULL`, tu se sačuva prethodno podešavanje
- Jedan handler za više signala: koji je stigao, kaže argument `sig`

Ko je poslao signal? SA_SIGINFO

```
void handler(int sig, siginfo_t *si, void *ctx) {
    posiljalac = si->si_pid;   // 0 ako je jezgro
    kod = si->si_code;         // SI_USER, ...
}

struct sigaction sa;
sa.sa_sigaction = handler;     // ne sa_handler!
sa.sa_flags = SA_SIGINFO;
sigemptyset(&sa.sa_mask);
sigaction(SIGINT, &sa, NULL);
```

ko šalje	si_code	si_pid
kill()	SI_USER	PID pošiljaoca
Ctrl+C	SI_KERNEL	0

Handler sa tri argumenta zna i **ko** je poslao signal.

Rijetko je potrebno: programu je obično svejedno odakle je signal stigao.

Šta smije da radi handler



- Posljedica: izmiješan ispis, pokvaren hip ili program koji se zaglavi (*deadlock*). Javlja se rijetko, pa se ne otkriva testiranjem
- U handleru se smiju pozivati samo **async-signal-safe** funkcije: `write`, `read`, `_exit`, `kill`, `waitpid`, `sigaction`…
- Nisu bezbjedne: `printf`, `malloc`, `free`, pa ni `exit`
- Spisak: `man 7 signal-safety`

Obrazac 1: zastavica

```c
volatile sig_atomic_t kraj = 0;

void handler(int sig) {
    kraj = 1;          // samo zastavica
}

int main() {
    /* sigaction(SIGTERM, ...) */
    while (!kraj) {
        radi_nesto();
    }
    pospremi();        // printf, free...
    return 0;
}
```

- Kad program poslije signala treba da **nastavi** rad
- Handler samo postavi zastavicu, a pravi posao radi `main`
- `volatile`: kompajler je mora čitati iz memorije, a ne držati u registru
- `sig_atomic_t`: upis jednom instrukcijom, pa ga signal ne prekida na pola

Primjer: `10_signali_terminal.c`

Obrazac 2: write i _exit

```c
void obrada_sigterm(int sig) {
    const char poruka[] = "Primio sam SIGTERM, izlazim.\n";
    write(STDOUT_FILENO, poruka, sizeof(poruka) - 1);
    _exit(0);
}
```

- Kad handler treba **odmah da završi** proces
- `write` je direktan sistemski poziv: nema bafer ni stanje u memoriji procesa
- `_exit` ne prazni `stdio` bafere, pa je bezbjedan; `exit` bi ih praznio, kao `printf`
- `sizeof(poruka)- 1`: dužina bez `'\0'`. Radi samo zato što je `poruka` niz, a ne pokazivač

Primjer: `09_signali_kill.c`

Signal prekida blokirajuće pozive

- Ako signal stigne dok proces čeka u `sleep`, `read`, `wait`, `pause`… i izvrši se handler, poziv se **prekida**
- `sleep(n)` se vrati ranije, sa brojem preostalih sekundi
- `read`, `waitpid`… vrate `-1`, a `errno` je `EINTR`
- Često korisno: u `10_signali_terminal.c` signal prekine `sleep(1)`, pa petlja odmah vidi zastavicu
- Kad prekid nije poželjan: `SA_RESTART` (jezgro samo ponovi `read`, `wait`…, ali ne i `sleep`) ili ručno:

```
ssize_t n;
do {
    n = read(fd, buf, sizeof buf);
} while (n == -1 && errno == EINTR);   // prekinut signalom: ponovi
```

Čekanje signala: alarm i pause

- `alarm(n)`: jezgro pošalje `SIGALRM` za `n` sekundi; `alarm(0)` ga poništava
- `pause()`: spava dok ne stigne signal

Primjer: korisnik ima 5 s da nešto otkuca. Handler je prazan, jer mu je jedini posao da prekine `read`. Bez njega bi `SIGALRM` završio proces.

```c
void na_alarm(int sig) { }   // samo prekine read

struct sigaction sa;
sa.sa_handler = na_alarm;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;             // bez SA_RESTART
sigaction(SIGALRM, &sa, NULL);

alarm(5);                    // za 5 s: SIGALRM
ssize_t n = read(STDIN_FILENO, buf, sizeof buf);
alarm(0);                    // stigao je: ponisti
if (n == -1 && errno == EINTR)
    printf("Isteklo vrijeme!\n");
```

Blokiranje signala

- Proces može privremeno da **blokira** signale, npr. dok radi nešto što ne smije biti prekinuto na pola
- Blokiran signal se **ne gubi**: čeka i isporuči se čim se odblokira
- `SIGKILL` i `SIGSTOP` se ne mogu blokirati

```c
sigset_t skup;
sigemptyset(&skup);
sigaddset(&skup, SIGINT);
sigprocmask(SIG_BLOCK, &skup, NULL);     // od sada Ctrl+C ceka
upisi_fajl();                            // ne smije se prekinuti
sigprocmask(SIG_UNBLOCK, &skup, NULL);   // SIGINT se sada isporuci
```

`SIG_SETMASK` postavlja cijelu masku; skup još mijenjaju `sigfillset`, `sigdelset`, `sigismember`.

Signali, fork i exec

	dijete poslije `fork`	program poslije `exec`
signal sa handlerom	isti handler	`SIG_DFL`: kod handlera više ne postoji
ignorisan signal	ignoriše ga	i dalje ga ignoriše
maska blokiranih	ista	ista
signali na čekanju	počinje bez njih	ostaju

- `nohup`: postavi `SIGHUP` na `SIG_IGN`, pa uradi `exec` traženog programa
- Mini shell koji ignoriše `SIGINT` mora u djetetu, prije `exec`-a, da vrati `SIG_DFL`; inače ni komande ne bi mogle da se prekinu sa Ctrl+C

SIGCHLD i zombiji

Kad dijete završi, jezgro roditelju pošalje `SIGCHLD` (podrazumijevano se ignoriše). Roditelj koji ne želi ni da čeka u `wait`, ni zombije, pokupi djecu u handleru:

```
void na_sigchld(int sig) {
    int sacuvan = errno;          // waitpid mijenja errno
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;                         // pokupi SVU djecu koja su zavrsila
    errno = sacuvan;
}
```

- Petlja je neophodna jer se signali ne broje: troje djece može dati jedan `SIGCHLD`
- `signal(SIGCHLD, SIG_IGN)`: nema zombija, ali ni izlaznog statusa djece

Primjer: kill iz programa

```
$ ./09_signali_kill ignor
[DIJETE] signal(SIGKILL): Invalid argument
[DIJETE] PID=4320, jos sam ziv...
[DIJETE] PID=4320, jos sam ziv...
[DIJETE] PID=4320, jos sam ziv...
[RODITELJ] saljem SIGTERM djetetu 4320
[DIJETE] PID=4320, jos sam ziv...
[DIJETE] PID=4320, jos sam ziv...
[DIJETE] PID=4320, jos sam ziv...
[RODITELJ] dijete jos zivi, saljem SIGKILL
[RODITELJ] dijete ubijeno signalom 9 (Killed)
```

- Bez argumenta: dijete hvata `SIGTERM` i završi sa `_exit(0)`
- `ignor`: dijete ignoriše `SIGTERM`, pa ga roditelj ubije sa `SIGKILL`
- `signal(SIGKILL, ...)` ne uspijeva
- Sa `| cat` poruke `jos sam ziv` nestanu. Zašto?

Primjer: signali iz terminala

Pokrenuti `./10_signali_terminal` (ispiše svoj PID), a `kill` slati iz drugog terminala.

akcija	šta se desi	$?
`kill -USR1 PID`	ispiše status i radi dalje	
Ctrl+C, prvi i drugi put	broji, radi dalje	
Ctrl+C, treći put	`Tri puta Ctrl+C, dosta je.`	0
`kill PID`	`Primio SIGTERM, izlazim uredno.`	0
`kill -9 PID`	shell ispiše `Killed`	137
Ctrl+\	`SIGQUIT` nije uhvaćen: `Quit`	131
Ctrl+Z, pa `fg`	`Stopped` (stanje `T`), pa nastavlja	

Uhvaćeni signali daju $? = 0, jer program sam završi.

Primjer: `10_signali_terminal.c`

04

Niti



Operativni sistemi

Šta je nit?

- **Nit** (*thread*): tok izvršavanja unutar procesa
- Proces ima bar jednu nit, onu koja izvršava `main`, a može ih imati više
- Na višejezgarnom procesoru niti rade **zaista paralelno**
- Pravljenje niti je jeftinije od pravljenja procesa: ne kopira se adresni prostor
- Niti komuniciraju preko zajedničkih promjenljivih, ali pristup mora da se sinhronizuje



Šta niti dijele

niti jednog procesa **dijele**	svaka nit ima **svoje**
kod, globalne promjenljive, hip otvorene fajlove, PID obradu signala, radni direktorijum	stek (lokalne promjenljive) registre i programski brojač ID niti (TID), `errno`

- Lokalna promjenljiva jedne niti je na njenom steku, ali druga nit može da joj pristupi ako dobije njenu adresu
- Greška u jednoj niti (npr. `SIGSEGV`) ruši **cijeli proces**

pthread: prevođenje i greške

- Sve funkcije su u `<pthread.h>`; program se prevodi sa opcijom `-pthread`

```
$ gcc -Wall -pthread 11_pthread_create_join.c -o 11_pthread_create_join
```

- `pthread_*` funkcije ne vraćaju -1: vraćaju `0`, ili kod greške
- Kod greške ne upisuju u `errno`, pa `perror` može ispisati pogrešnu poruku

```
int greska = pthread_create(&nit, NULL, posao, NULL);
if (greska != 0) {
    fprintf(stderr, "pthread_create: %s\n", strerror(greska));
    exit(1);
}
```

Još: `pthread_self()` (ID tekuće niti), `pthread_detach(nit)` (nit se sama oslobodi, bez `join`-a), `pthread_cancel(nit)` (zahtjev za prekid).

pthread_create

```
int pthread_create(pthread_t *nit, const pthread_attr_t *attr,
                   void *(*funkcija)(void *), void *arg);
```

`nit`	gdje da upiše ID nove niti
`attr`	atributi (veličina steka…); najčešće `NULL`
`funkcija`	funkcija koju nit izvršava: prima i vraća `void *`
`arg`	argument koji ta funkcija dobija

void * nije niz argumenata

`void *` je **jedan pokazivač bez tipa**: može da pokazuje na bilo šta, ali ne zna na šta. Biblioteka ne zna kakve podatke tvoja nit treba, pa traži pokazivač „na bilo šta“.

Argument niti

```
void *posao(void *arg) {
    int id = *(int *)arg;
    printf("Nit %d\n", id);
    return NULL;
}
```

- Pri pozivu cast ne treba: svaki pokazivač se sam pretvara u `void *`
- U funkciji treba, jer se `void *` ne može dereferencirati

```
// POGRESNO: sve niti dobijaju &i
for (int i = 0; i < N; i++)
    pthread_create(&niti[i], NULL,
                   posao, &i);

// ISPRAVNO: svaka nit svoj element
int ids[N];
for (int i = 0; i < N; i++) {
    ids[i] = i;
    pthread_create(&niti[i], NULL,
                   posao, &ids[i]);
}
```

Nit počne **kasnije**, pa sa `&i` pročita vrijednost koju `i` ima *tada*.

Više argumenata: struktura

Funkcija niti prima **tačno jedan** argument. Više podataka se pakuje u **strukturu**, a niti se šalje pokazivač na nju.

```c
struct argumenti {
    int id, koraci;
    const char *ime;
};
void *posao(void *arg) {
    struct argumenti *a = arg;
    for (int i = 0; i < a->koraci; i++)
        printf("%s %d: %d\n", a->ime, a->id, i);
    return NULL;
}
```

```c
struct argumenti args[N];
for (int i = 0; i < N; i++) {
    args[i].id = i;
    args[i].koraci = 3;
    args[i].ime = "Nit";
    pthread_create(&niti[i], NULL,
                   posao, &args[i]);
}
```

Kao kod `ids[]`: svaka nit dobija svoju strukturu, koja postoji dok je nit ne pročita.

Povratna vrijednost: pthread_exit i pthread_join

```c
void *zbir(void *arg) {
    long n = (long)arg;     // broj u void*
    long *rez = malloc(sizeof(long));  // hip!
    *rez = 0;
    for (long i = 1; i <= n; i++) *rez += i;
    pthread_exit(rez);      // = return rez;
}

pthread_create(&n1, NULL, zbir, (void *)100L);
long *r1;
pthread_join(n1, (void **)&r1);  // ceka
printf("%ld\n", *r1);            // 5050
free(r1);
```

- `pthread_join`: čeka da nit završi (kao `waitpid`) i preuzme njen rezultat
- `pthread_exit`: završava **tekuću nit**
- Nit koju niko ne sačeka zadržava resurse, kao zombi

Ne vraćati adresu lokalne promjenljive: stek niti nestaje kad ona završi.

Primjer: niti jednog procesa

```
$ ./11_pthread_create_join
Nit 0 (PID=7000) korak 0
Nit 1 (PID=7000) korak 0
main: napravio 4 niti, cekam ih...
# u drugom terminalu:
$ ps -L -p 7000
    PID     LWP TTY          TIME CMD
   7000    7000 pts/1    00:00:00 11_pthread_crea
   7000    7001 pts/1    00:00:00 11_pthread_crea
   7000    7002 pts/1    00:00:00 11_pthread_crea
   7000    7003 pts/1    00:00:00 11_pthread_crea
   7000    7004 pts/1    00:00:00 11_pthread_crea
```

- Sve niti ispisuju **isti PID**; `ps -L` prikaže jedan proces sa pet niti
- Kolona `LWP` je ID niti; glavna nit ima ID jednak PID-u
- Redoslijed koraka se mijenja od pokretanja do pokretanja

Niti dijele memoriju

Isti eksperiment kao u `02`: druga strana mijenja globalnu i promjenljivu na hipu.

```
$ ./02_adresni_prostor              # procesi
[DIJETE]   globalna=110 (0x5f5e30cd8010)
[DIJETE]   dinamicka=130 (0x5f5e5a88f010)
[RODITELJ] globalna=10 (0x5f5e30cd8010)
[RODITELJ] dinamicka=30 (0x5f5e5a88f010)
$ ./12_niti_dijele_memoriju         # niti
[NIT]  globalna=110 (0x563aeec7b010), dinamicka=130 (0x563b1a48c010)
[MAIN] globalna=110 (0x563aeec7b010), dinamicka=130 (0x563b1a48c010)
```

- Proces: dijete mijenja **svoju kopiju**, roditelj vidi stare vrijednosti
- Nit: `main` vidi izmjene, jer je memorija jedna; zato i jedan `free` umjesto dva

Primjer: `02_adresni_prostor.c, 12_niti_dijele_memoriju.c`

return ili pthread_exit u main-u

```
$ ./14_exit_vs_pthread_exit
Nit radi... 1/5
Nit radi... 2/5
main: return - gasi se cijeli proces
```

```
$ ./14_exit_vs_pthread_exit niti
Nit radi... 1/5
Nit radi... 2/5
main: pthread_exit - nit nastavlja
Nit radi... 3/5
Nit radi... 4/5
Nit radi... 5/5
Nit je zavrsila svoj posao.
```

- `exit()` ili `return` iz `main`-a završava **cijeli proces**, sa svim nitima
- `pthread_exit()` u `main`-u završava samo glavnu nit; proces živi dok ne završi i posljednja nit
- Uobičajeno je da `main` sačeka niti sa `pthread_join`

Primjer: `14_exit_vs_pthread_exit.c`

sched_yield: ustupanje procesora

- `sched_yield()`: nit **dobrovoljno ustupa procesor** i prelazi u stanje „spremna“
- Na višejezgarnom procesoru niti rade paralelno, pa nema kome da ustupi procesor
- Zato primjer veže proces za jezgro 0 (`sched_setaffinity`); spolja: `taskset -c 0 ./program`

```
$ ./15_sched_yield
Prvih 200 koraka:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
Ukupno smjena niti: 7 (od 4000 koraka), yield=NE
$ ./15_sched_yield yield
Prvih 200 koraka:
ABABABABABABABABABABABABABABABABABABABABABABABABABABABAB...
Ukupno smjena niti: 3998 (od 4000 koraka), yield=DA
```

Bez yield-a nit radi dok joj ne istekne kvantum; sa yield-om ustupa procesor poslije svakog koraka.

Trka za podacima

`brojac++` nije jedna, nego **tri** operacije: pročitaj u registar, uvećaj, upiši.

nit 1	nit 2	brojac
pročita 5		5
	pročita 5	5
upiše 6		6
	upiše 6	**6**

Dva uvećanja, a `brojac` je porastao za jedan.

```c
long brojac = 0;      // zajednicki

void *uvecaj(void *arg) {
    for (int i = 0; i < 1000000; i++)
        brojac++;
    return NULL;
}
```

```
$ ./16_trka_niti
Ocekivano: 2000000
Dobijeno:  1003601
```

Trka za podacima (*race condition*). Kod koji pristupa zajedničkom podatku je **kritična sekcija**. Rješenje, mutex i semafori, tema je sljedećih vježbi.

05

Pregled

Proces ili nit

	proces: `fork()`	nit: `pthread_create()`
adresni prostor	kopija, odvojen	zajednički
identifikator	novi PID	isti PID, novi TID
cijena pravljenja	veća	manja
komunikacija	potreban IPC: pipe, signali, dijeljena memorija	preko zajedničkih promjenljivih
čekanje	`wait`, `waitpid`	`pthread_join`
završetak	`exit`	`pthread_exit`, `return`
greška u jednom	ne ruši ostale	ruši cijeli proces
problem	–	trka za podacima: mutex, semafori

Ispod haube: sistemski poziv clone

Na Linuxu i proces i nit nastaju istim sistemskim pozivom, `clone`. Zastavice određuju šta dijete **dijeli** sa roditeljem, a šta dobija **svoje**.

šta se pravi	glavne zastavice	šta dijete dobija
proces: `fork()`	samo `SIGCHLD`	kopiju svega
nit: `pthread_create()`	`CLONE_VM`, `CLONE_FILES`, `CLONE_FS`, `CLONE_SIGHAND`, `CLONE_THREAD`, …	dijeli memoriju, fajlove i obradu signala
kontejner (Docker)	`CLONE_NEWPID`, `CLONE_NEWNS`, `CLONE_NEWUTS`, `CLONE_NEWNET`, …	svoje PID-ove, montiranja, ime računara i mrežu

Proces, nit i kontejner su isti mehanizam jezgra; razlikuju se samo zastavice (`man 2 clone`).

Korisne komande

`ps -ef`	svi procesi
`ps -o pid,ppid,stat,cmd`	izabrane kolone; `STAT` je stanje
`ps -L -p PID`	niti procesa
`pstree -p`	stablo procesa
`top, top -H`	procesi, odnosno niti, uživo
`echo $?`	izlazni kod posljednje komande

`kill -l`	spisak signala
`kill -SIGNAL PID`	slanje signala
`pkill ime`	signal po imenu programa
`jobs, fg, bg`	poslovi u shell-u
`taskset -c 0 ./prog`	samo na jezgru 0
`nproc`	broj jezgara
`man 2 fork`	2: sistemski pozivi, 3: biblioteka, 7: teme

Hvala na pažnji

Pitanja?

UCG
Univerzitet Crne Gore